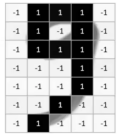
Convolutional Neural Networks

Jacques Bahi

Classification in Computer vision


-1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1


-1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1

Location shifted

To handle variety in digits we can use simple artificial neural network (ANN)

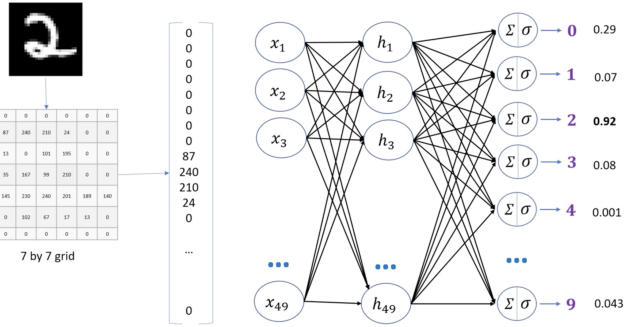

0 0 0 0 0 0 0
0 87 240 210 24 0 0
0 13 0 101 195 0 0
0 35 167 99 210 0 0
0 145 230 240 201 189 140
0 0 102 67 17 13 0
0 0 0 0 0 0 0
7 by 7 grid
0
0
0
0
0
0
0
0
87
240
210
24
0
...
0
x_1
x_2
x_3
x_{49}
h_1
h_2
h_3
h_{49}
Σ σ
0 0.29
1 0.07
2 0.92
3 0.08
4 0.001
9 0.043
49*49 = 2401
49*10 = 490

Image size = 1920 x 1080 X 3

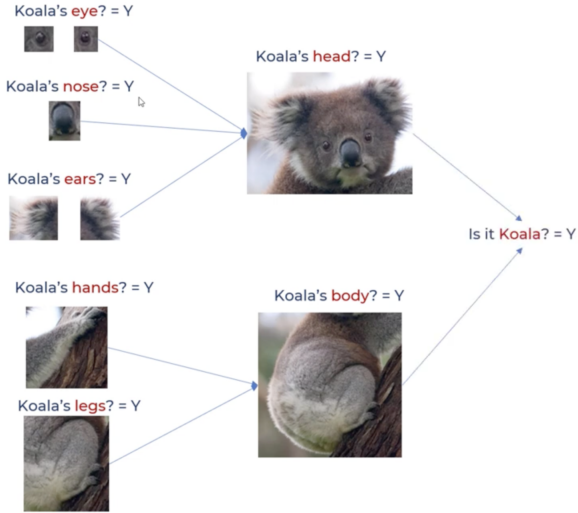
First layer neurons = 1920 x 1080 X 3 ~ 6 million

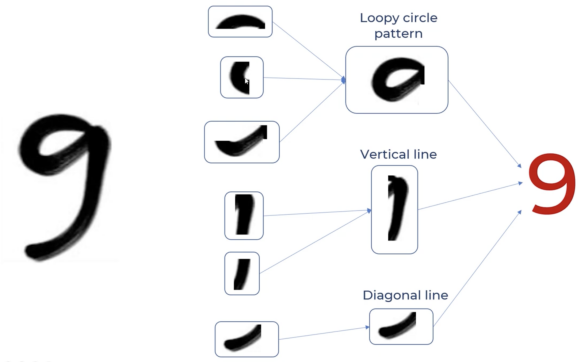
Hidden layer neurons = Let's say you keep it ~ 4 million

Weights between input and hidden layer = 6 mil * 4 mil
= 24 million

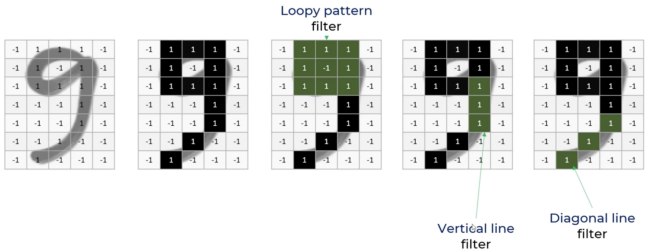
Disadvantages of using ANN for image classification

1. Too much computation
2. Treats local pixels same as pixels far apart
3. Sensitive to location of an object in an image


Koala's eye? = Y
Koala's nose? = Y
Koala's ears? = Y
Koala's head? = Y
Is it Koala? = Y
Koala's hands? = Y
Koala's legs? = Y
Koala's body? = Y


Loopy circle pattern
Vertical line
Diagonal line
9

1. Filters (Kernels)



-1	**1**	**1**	**1**	-1
-1	**1**	-1	**1**	-1
-1	**1**	**1**	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	**1**	-1	-1
-1	**1**	-1	-1	-1

1	1	1
1	-1	1
1	1	1

-1+1+1-1-1-1-1+1+1 = -1 → -1/9 = -0.11

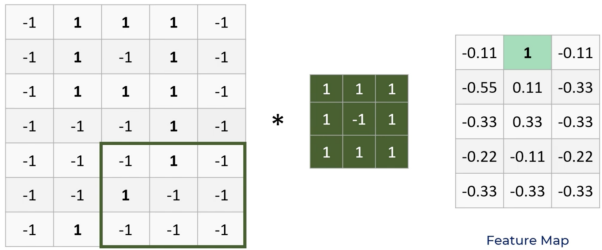
-1	**1**	**1**	**1**	-1
-1	**1**	-1	**1**	-1
-1	**1**	**1**	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	**1**	-1	-1
-1	**1**	-1	-1	-1

*

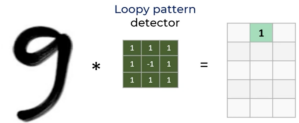
1	1	1
1	-1	1
1	1	1

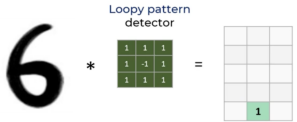
-0.11		

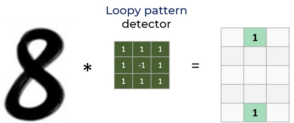
2. Stride

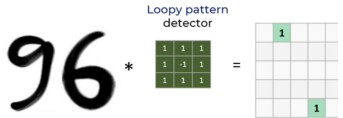

-1 1 1 1 -1
-1 1 -1 1 -1
-1 1 1 1 -1
-1 -1 -1 1 -1
-1 -1 -1 1 -1
-1 -1 1 -1 -1
-1 1 -1 -1 -1
*
1 1 1
1 -1 1
1 1 1
-0.11 1 -0.11
-0.55 0.11 -0.33
-0.33 0.33 -0.33
-0.22 -0.11 -0.22
-0.33 -0.33 -0.33
Feature Map


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
1


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
1

Location invariant: It can detect eyes in any location of the imag

*

eye
detector

=

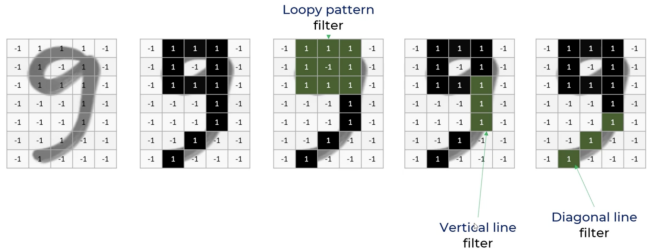
eye
detector

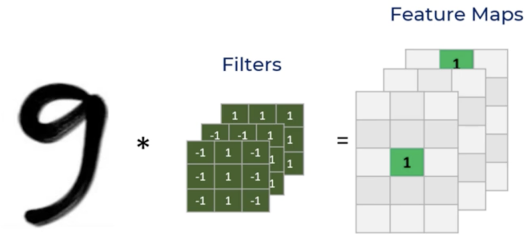
=

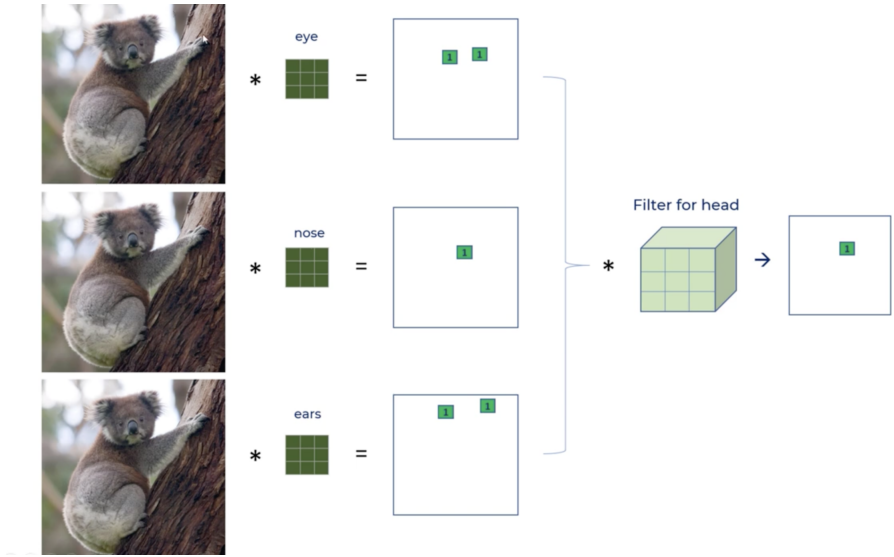
eye
detector

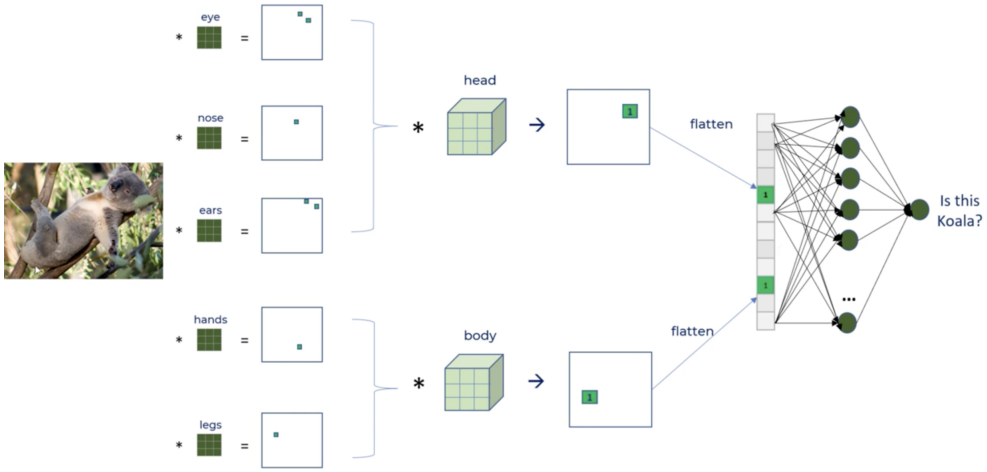
*

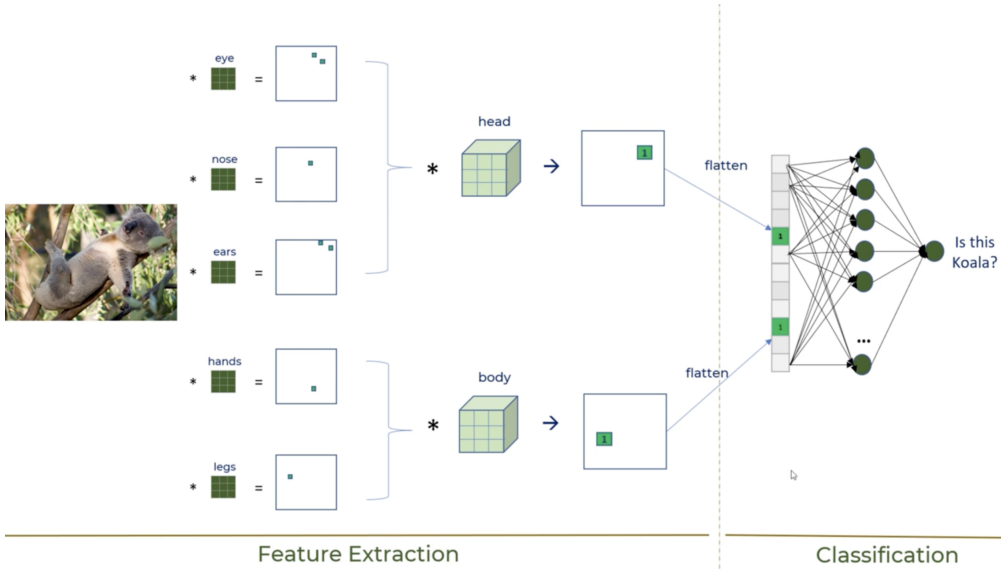
=


Loopy pattern filter
Vertical line filter
Diagonal line filter


Feature Maps
Filters
*
=
1 1 1
-1 -1 1
-1 1 -1
-1 1 -1
-1 1 -1
1
1


eye
nose
ears
Filter for head
1
1
1
1
1
1
1


eye
nose
ears
hands
legs
head
body
flatten
flatten
Is this Koala?


eye
nose
ears
hands
legs
head
body
flatten
flatten
Is this Koala?
Feature Extraction
Classification

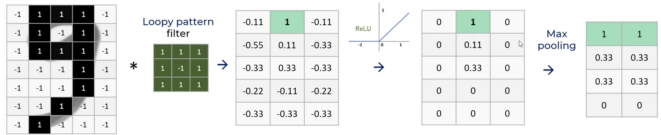

Loopy pattern filter
ReLU

3. Pooling

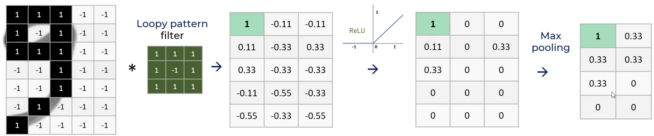
5	1	3	4
8	2	9	2
1	3	0	1
2	2	2	0

8	**9**
3	2

2 by 2 filter with stride = 2


Loopy pattern filter
ReLU
Max pooling

Shifted 9 at different position

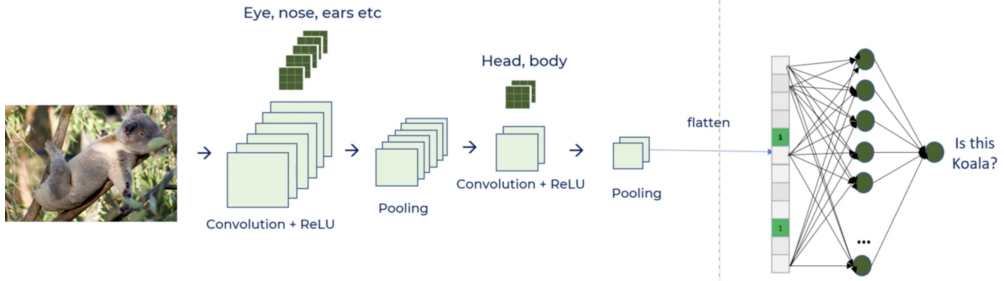


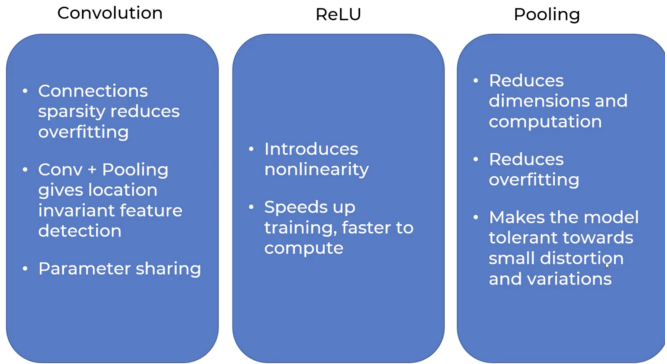
Benefits of pooling

Reduces dimensions & computation

Reduce overfitting as there are less parameters

Model is tolerant towards variations, distortions


Eye, nose, ears etc
Head, body
flatten
Convolution + ReLU
Pooling
Convolution + ReLU
Pooling
Is this Koala?
Feature Extraction
Classification


Convolution
• Connections sparsity reduces overfitting
• Conv + Pooling gives location invariant feature detection
• Parameter sharing
ReLU
• Introduces nonlinearity
• Speeds up training, faster to compute
Pooling
• Reduces dimensions and computation
• Reduces overfitting
• Makes the model tolerant towards small distortion and variations

4. Padding

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

a

1	2	3
6	7	8
11	12	13

b

3	4	5
8	9	10
13	14	15

a	b
c	d

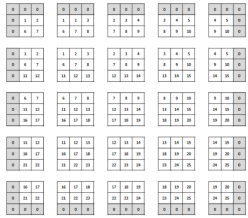
c

11	12	13
16	17	18
21	22	23

d

13	14	15
18	19	20
23	24	25

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0



Convolution Layer Hyperparameters

The main hyperparameters of a convolutional layer are :

1. the filter window size,
2. the number of filters,
3. the stride,
4. the padding.

- The size of the window (window_height $\times$ window_width) containing information from spatially close pixels is usually 3×3 or 5×5.
- The number of filters, indicating the number of features we want to handle (output_depth), is typically 32 or 64.
- In Keras Conv2D layers, these hyperparameters are to be specified as arguments in the following order : Conv2D(output_depth, (window_height, window_width))